A TCP Socket-Based Multi-User Messaging System with a Graphical Client and Reliable File Transfer

Li Haolin
The Education University of Hong Kong
Email: s1153657@s.eduhk.hk

Lam Chun Kit
The Education University of Hong Kong
Email: s1154051@s.eduhk.hk

Liu Chenghao
The Education University of Hong Kong
Email: s1153655@s.eduhk.hk

***Abstract*—This paper presents the design and implementation of a TCP socket-based multi-user messaging system in Python. The system follows a client-server architecture and supports the core functions of a message application, including authentication, friend list management, direct and broadcast messaging, text-file delivery, inbox operations, and acknowledgement generation. Server-side data, including user records, friend lists, messages, and files, are persistently maintained in JSON format, while communication is handled through a lightweight custom application-layer protocol.**

To extend the baseline project requirements, the system incorporates a Tkinter-based graphical client, online/offline friend status display, and an enhanced file-transfer mechanism with chunked transmission, SHA-256 integrity verification, optional compression, and progress-bar feedback. These additions improve both usability and transfer robustness while preserving a simple and readable implementation structure. The final system provides a compact but complete demonstration of socket programming, protocol design, persistent messaging services, and practical reliability enhancements in a student-level network application.

I. Introduction

Socket programming is a fundamental topic in internetworking because it links transport-layer communication with the design of application-layer protocols and client-server systems. To explore these concepts in a practical setting, this project implements a TCP socket-based multi-user messaging system in Python. The system adopts a client-server architecture and a lightweight custom text-based protocol to support user communication, persistent message storage, and file delivery over the network.

Beyond the baseline functions required by the project specification, the final system extends the messaging platform with user registration, online/offline friend status display, a Tkinter-based graphical client, and a more reliable file-transfer workflow based on chunked transmission, SHA-256 integrity verification, optional compression, and progress feedback. These enhancements improve both usability and transfer robustness while keeping the implementation compact and easy to inspect. The remainder of this paper describes the system architecture, details the steps required to run the client and server applications, and presents the protocol specification used in the implementation.

TABLE I
Main Project Components

File / Directory	Main Responsibility
`src/config.py`	Defines network parameters and transfer-related settings, such as host, port, buffer size, chunk size, and compression threshold.
`src/protocol.py`	Implements request/response formatting, field escaping and restoration, and helper functions for file-transfer processing.
`src/storage.py`	Handles persistent data management, including user records, friend lists, inbox items, file records, and acknowledgement entries.
`src/server.py`	Implements the core server logic, including socket setup, client-session handling, command validation, online-status tracking, and transfer reassembly.
`src/client.py`	Provides a basic command-line client for compatibility testing and simple manual interaction with the server.
`src/gui_client.py`	Provides the main Tkinter-based graphical client for login, messaging, file transfer, inbox access, and transfer progress display.
`data/`	Stores persistent JSON data files used by the server, including default user information and server-side messaging data.
`text_files/`	Contains sample text files used for file-transfer testing and demonstration scenarios.

II. System Overview

A. Project Structure

The implementation is divided into a set of modules for configuration, protocol handling, storage, server-side control, and client-side interaction. Table I summarizes the role of each main component.

B. Design Goals

The system design is guided by the following principles:

- **Functional completeness:** The implementation should fully cover the baseline project requirements through a coherent client-server architecture for messaging, file transfer, inbox operations, and acknowledgement handling.
- **Protocol and storage clarity:** The application-layer protocol and storage model should remain lightweight and

inspectable, so that command formats, validation logic, and persistent data organization can be clearly understood and verified.

- **Usability for demonstration:** The client interface should support reliable demonstration and convenient interaction, with the Tkinter GUI acting as the primary interface and the CLI client preserved for compatibility and manual testing.
- **Practical enhancement:** The system should incorporate meaningful extensions beyond the baseline specification, particularly in transfer reliability and usability, while preserving readability, modularity, and maintainability.

III. Architecture and Implementation

A. Server Side

The server is implemented in `src/server.py` as the central control component of the system. It uses Python TCP sockets to accept client connections and creates one worker thread for each connected client, allowing multiple users to interact with the messaging service concurrently. For each request, the server parses the command, validates the sender identity and parameters, applies the required business logic, updates persistent data through the storage layer, and returns a compact success or error response to the client.

In addition to handling the baseline messaging workflow, the server is responsible for maintaining online session state and coordinating the enhanced file-transfer process. Online users are tracked through an in-memory connection map, which allows the system to support reconnect behavior and provide explicit online/offline status information to the GUI client. For chunked file transfer, the server also manages temporary transfer state, receives incoming chunks, performs reassembly and verification, and converts the final payload into a normal stored inbox item after successful validation.

B. Storage Layer

Persistent storage is managed through JSON files located under `data/`, primarily `default_users.json` and `server_data.json`. This layer stores user records, friendship relationships, messages, files, and acknowledgement entries in a human-readable format. Such a design is intentionally lightweight and inspectable, making the stored system state easy to verify during development, testing, and project demonstration.

The storage model is organized around logical messaging data rather than transport-specific representations. In particular, even when a file is transmitted using compression and chunking, the final saved content is restored to normal text form before being written into persistent storage. This keeps the transport mechanism separated from the long-term data representation and avoids introducing unnecessary complexity into the persistence layer.

C. Client Side

The project provides two client interfaces with different purposes. The command-line client in `src/client.py` preserves the original menu-driven interaction model and serves mainly as a compatibility and manual testing interface. It is useful for verifying baseline protocol behavior without relying on graphical components.

The primary demonstration interface is the Tkinter-based GUI client in `src/gui_client.py`. It supports the main user workflow, including login, registration, friend-list operations, online/offline status refresh, direct messaging, broadcast messaging, file transfer, inbox access, and reconnect handling. In addition, the GUI exposes the enhanced transfer workflow more clearly by providing visible status updates and progress-bar feedback during large file transmission. This makes the client more suitable for live presentation while preserving consistency with the same server-side protocol and storage model.

D. Current Transfer Settings

The current implementation uses the configuration values shown in Table II.

TABLE II
Current Transfer Settings

Parameter	Value	Description
Server host	`0.0.0.0`	Server bind address
Client host	`127.0.0.1`	Default client connection target
Port	`12345`	TCP service port
Socket buffer size	`1024`	Receive/send buffer size used by the socket logic
Transfer chunk size	`700`	Maximum payload size of each transfer chunk
Compression threshold	`256` bytes	Minimum payload size for applying compression

IV. Detailed Running Instructions

This section is based on the actual current project structure and was verified in an environment with Python 3.13.3. The project uses only Python standard-library modules.

A. Environment and Files

Before running the project, confirm the following items:

- Python 3 is installed and available as `python3`.
- The project root contains `src/`, `data/`, `docs/`, and `text_files/`.

B. Start the Server

Open a terminal in the project root and run:

```
python3 src/server.py
```

The server listens on port `12345`. Keep this terminal open while using any client.

C. Start the GUI Client

Open another terminal in the project root and run:

```
python3 src/gui_client.py
```

For a two-user demonstration, open a second GUI client in another terminal and log in as a different user.

D. Default Accounts and Registration

The project already includes 10 default users as required in `data/default_users.json`. New accounts can also be created from the GUI login window with the Register function.

E. Practical Demo Flow

The most reliable demonstration sequence is:

1) start the server;
2) open two GUI clients;
3) log in as two different users;
4) add each other as friends if needed;
5) refresh the friend list and friend status;
6) send a normal text message;
7) send a broadcast message;
8) send a small text file such as `text_files/sample1.txt`;
9) send a large text file such as `text_files/sample3.txt` to trigger compression and multiple chunks;
10) open the receiver inbox and read the incoming items.

F. Role of the CLI Client

The CLI client still exists and can be started with:

```
python3 src/client.py
```

Its role is compatibility and simple manual testing. The final project demonstration should focus on the GUI because the progress bar and chunked transfer workflow are implemented there with the least code disruption.

V. Protocol Specification

A. General Message Format

The implementation uses a lightweight text-based application-layer protocol over TCP. Each request begins with a command name followed by one or more parameters separated by the vertical bar character `|`. For commands that involve multiple recipients, user IDs are encoded as a comma-separated list inside a single field. To preserve protocol compatibility, free-text content is escaped before transmission and restored after parsing by helper functions in `src/protocol.py`.

Most server replies follow a uniform reply structure:

`OK|COMMAND|message`

for successful processing, and

`ERROR|COMMAND|message`

for invalid requests or failed validation. Some commands, such as `READ_ITEM`, return additional structured fields after the command name in order to carry message or file metadata together with the item content. This design keeps the protocol easy to inspect while still supporting the full messaging workflow.

B. Core Commands

Table III summarizes the main commands used in the baseline messaging workflow.

TABLE III
Core Protocol Commands

Command	Request Format	Main Rules / Behavior
LOGIN	LOGIN|user_id|password	Authenticates an existing user. The account must exist and the password must match. A successful request returns a standard OK reply and marks the user as logged in.
REGISTER	REGISTER|user_id|password	Creates a new account. The user ID must contain lowercase letters or digits only, must not already exist, and the password must be non-empty.
VIEW_FRIENDS	VIEW_FRIENDS|user_id	Returns the caller's friend list. The requester must already be logged in and may only request its own data. If no friends exist, the server returns NO_FRIENDS.
VIEW_FRIENDS_STATUS	VIEW_FRIENDS_STATUS |user_id	Returns the caller's friend list together with online/offline state derived from current server-side session tracking. The same identity checks as VIEW_FRIENDS apply.
ADD_FRIEND	ADD_FRIEND|user_id|friend_id	Adds an existing user to the caller's friend list. Both users must exist, self-addition is rejected, and duplicate friendship is not allowed.
DELETE_FRIEND	DELETE_FRIEND |user_id|friend_id	Removes an existing friend from the caller's friend list. The requester must be logged in and the target must already be present in the current friend list.
SEND_MESSAGE	SEND_MESSAGE |sender|r1,r2|content	Sends a text message to one or more valid friends. The sender must be logged in, recipients must be existing friends, and self-sending is rejected. Multi-line content is supported after escaping.
BROADCAST	BROADCAST|sender|content	Sends the same message to all friends of the sender. The sender must be logged in and must already have at least one friend.
SEND_FILE	SEND_FILE |sender|r1,r2| file_name|file_content	Sends a text file to one or more valid friends. The sender must be logged in, recipients must be friends, the file must use the .txt extension, and the content must not be empty.
LIST_INBOX	LIST_INBOX|user_id	Returns a summary view of the caller's inbox. The requester must be logged in and may only access its own inbox.
READ_ITEM	READ_ITEM |user_id|item_number	Returns the full content of the selected inbox item together with its metadata. The item number must be a valid positive index. Reading a direct message or file also triggers acknowledgement generation for the original sender.
DELETE_ITEM	DELETE_ITEM |user_id|item_number	Deletes the selected inbox item. The same index validation rules as READ_ITEM apply.
LOGOUT	LOGOUT	Ends the current authenticated session and returns a standard logout confirmation reply. This command is valid only after login.

C. Acknowledgement Behavior

Acknowledgement is implemented as a normal inbox item generated by the server. When a user reads a direct message or file item, the server creates an acknowledgement entry in the original sender's inbox indicating that the item has been read. Broadcast items and acknowledgement items do not trigger further acknowledgement generation, which prevents recursive acknowledgement chains and keeps the inbox behavior predictable.

D. Chunked Transfer Extension

The final version extends the baseline file-transfer workflow with two additional protocol commands dedicated to GUI-based large-file delivery: `TRANSFER_START` and `TRANSFER_CHUNK`. The `TRANSFER_START` request carries transfer metadata, including the sender identity, recipient list, transfer identifier, file name, total chunk count, compression flag, and checksum information. The `TRANSFER_CHUNK` request carries the indexed payload chunks associated with that transfer. Together, these two commands allow the server to validate transfer metadata, accept and track chunked input, reject invalid or duplicate chunks, reconstruct the original payload, verify integrity, perform decompression when required, and then save the final file through the normal inbox and storage path.

The transfer sequence in the current implementation is:

1) read the text file in the GUI client;
2) encode the file content as UTF-8 text;
3) compress it with `zlib` when the content size is at least `256` bytes [1];
4) compute a SHA-256 checksum on the transfer bytes [2];
5) Base64-encode the transfer bytes;
6) split the Base64 payload into chunks of up to `700` characters;
7) send one `TRANSFER_START` request and then all `TRANSFER_CHUNK` requests;
8) reassemble the chunks on the server, verify checksum, decompress if needed, decode the original text, and save the final file item.

VI. Key Features and Enhancements

A. Baseline Functional Coverage

The implemented system covers the full baseline workflow required for the messaging application project:

- user login and logout;
- friend-list viewing, addition, and deletion;
- direct message sending to one or more friends;
- broadcast messaging to all friends;
- text-file sending;
- inbox listing, item reading, and item deletion;
- acknowledgement generation after a message or file is read;
- persistent server-side storage of user data, friend lists, messages, and files;
- support for the default set of preconfigured users required at server startup.

B. Project Highlights

Beyond the baseline specification, the final version introduces several enhancements that improve usability, transfer reliability, and overall system completeness. The main additions are summarized in Table IV.

TABLE IV
Major Enhancements and Practical Value

Enhancement	Practical Value
User registration	Removes dependence on a fully fixed user set and makes the messaging system more complete and flexible for demonstration.
GUI client	Provides a more accessible and presentation-friendly interface than the CLI workflow, especially for live project demonstration.
Friend-status display	Exposes online/offline presence using server-side session tracking and makes user interaction more informative.
Chunked file transfer (`TRANSFER_START`, `TRANSFER_CHUNK`)	Introduces application-layer segmentation and server-side reassembly for large payload delivery, while also making the transfer workflow more realistic and explainable.
Integrity verification (SHA-256)	Detects damaged or incomplete transferred data before final storage and improves confidence in the correctness of received file content.
Optional compression (`zlib`)	Reduces the size of large payloads before transmission and demonstrates an efficiency-oriented design choice for file delivery.
Progress feedback in GUI	Provides visible status updates during file sending and makes the transfer process easier to observe, explain, and demonstrate.

C. Why the Final Enhancements Matter

The final enhancements are important for both networking demonstration and user experience. Chunked transfer makes the protocol more realistic by showing how large application data can be divided into smaller units and reassembled at the application layer. Integrity verification adds a necessary validation step, ensuring that transferred content is checked before it is accepted and stored. Compression introduces an efficiency-oriented design decision for large payloads and demonstrates that transfer behavior can be adapted according to message size.

From the usability perspective, the GUI client, friend-status display, and progress feedback make the system easier to demonstrate and easier to understand during live testing. Instead of hiding protocol activity behind a command-line workflow, the final version exposes more of the system state and transfer process to the user. As a result, the enhancements do not merely add extra features; they also strengthen the project as a complete and explainable internetworking application.

VII. GUI Design Overview

The graphical user interface is implemented with Tkinter and serves as the primary client for project demonstration. Compared with the command-line client, the GUI presents the system workflow in a more accessible and observable form, especially for login, friend-list interaction, message sending, file transfer, and inbox access. The interface is intentionally kept simple so that the relationship between user actions and underlying protocol behavior remains easy to explain.

A. Login Window

The login window provides the entry point to the system. It supports user authentication, new-user registration, and reconnect-related interaction in a compact layout. This design allows the demonstration to begin from a clear authentication stage and makes it easier to show both the baseline login workflow and the registration enhancement introduced in the final version.

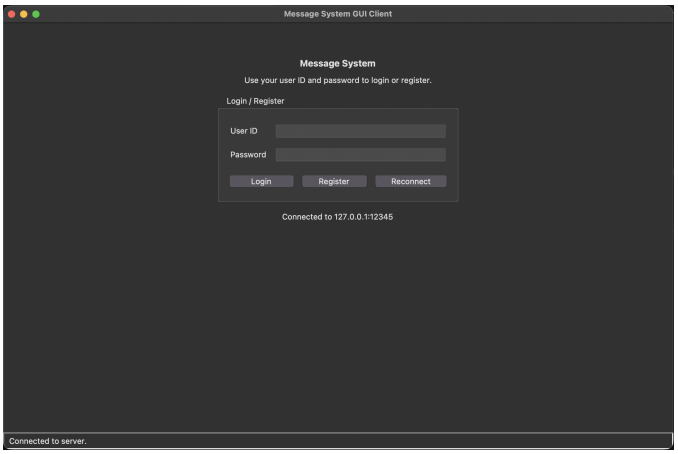

Fig. 1. Login window of the GUI client.

B. Main Layout

After successful login, the main window presents the core messaging workflow in a single integrated interface. It shows the current user, connection state, and friend-list area, together with tab-based working panels for direct messaging, broadcast messaging, file transfer, and inbox operations. The file-transfer tab also includes visible status updates and progress feedback, making the enhanced transfer mechanism easier to observe during live demonstration.

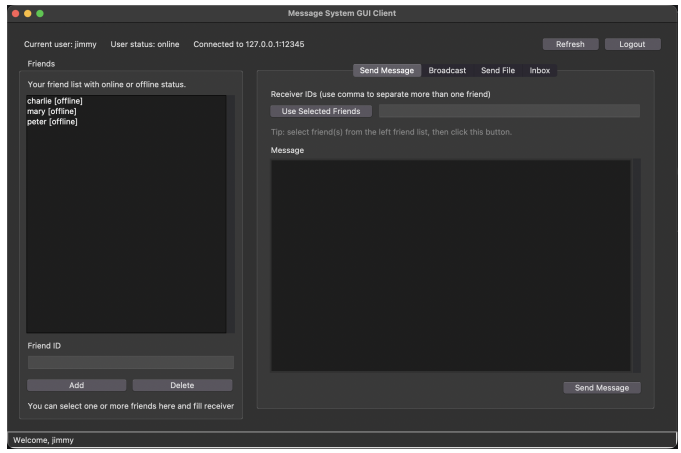

Fig. 2. Main layout of the GUI client after login.

VIII. Conclusion

This project presents a complete TCP socket-based multi-user messaging system implemented in Python with a client-server architecture, persistent server-side storage, and a lightweight custom application-layer protocol. The final system satisfies the baseline project requirements, including authentication, friend-list management, direct and broadcast messaging, text-file delivery, inbox operations, and acknowledgement generation.

Beyond the baseline workflow, the project incorporates several practical enhancements, including user registration, online/offline friend-status display, a Tkinter-based graphical client, and a more reliable file-transfer mechanism based on chunked transmission, SHA-256 integrity verification, optional compression, and progress feedback. These additions improve both usability and transfer robustness while keeping the overall design clear and inspectable.

Overall, the project demonstrates the integration of socket programming, protocol design, persistent data management, and graphical user interaction in a coherent internetworking application. It therefore provides not only a functional message system, but also a concrete demonstration of how application-layer services can be designed, implemented, and explained in a student-level networking project.

References

[1] Python Software Foundation, "zlib — compression compatible with gzip," Python 3 Documentation, 2025, https://docs.python.org/3/library/zlib.html.

[2] ——, "hashlib — secure hashes and message digests," Python 3 Documentation, 2025, https://docs.python.org/3/library/hashlib.html.